Real-Time ETL Architecture using Logstash

Mangesh Walimbe

Problem Statement

- High-frequency ingestion of candidate and assessment data
- Need for real-time candidate search capabilities
- Assessment score computation required for analytics
- Traditional ETL solutions introduce latency and complexity
- Requirement: lightweight, near real-time processing pipeline

Architecture Overview

PostgreSQL serves as the system of record

Logstash performs extraction, transformation, and routing

Elasticsearch enables real-time search and indexing

Kibana provides visualization and analytics

Pipeline operates in near real-time with incremental updates

Pipeline Design

Candidate Search Pipeline → optimized for query and retrieval

Assessment Analytics Pipeline → optimized for scoring and reporting

Decoupling improves scalability and maintainability

Independent index strategies per pipeline

Logstash JDBC Input

- Uses JDBC plugin to connect to PostgreSQL
- Supports incremental ingestion using tracking column
- Efficient queries reduce database load
- Example: WHERE updated_at > :sql_last_value

Input - Example

```
input {
    jdbc {
        jdbc_driver_library => "c:/logstash/jdbc/postgresql.jar"
        jdbc_driver_class => "org.postgresql.Driver"
        jdbc_connection_string => "${JDBC_HOST}"
        jdbc_user => "${DB_USER}"
        jdbc_password => "${DB_PWD}"
        jdbc_paging_enabled => true
        jdbc_page_size => 1000
        schedule => "* * * * *"  # schedule to run every minute
        statement => "SELECT * FROM employee WHERE updated_at > :sql_last_value"
        use_column_value => true
        tracking_column => "updated_at"
        tracking_column_type => "timestamp"
        last_run_metadata_path => "c:/logstash/employee.tracker"
    }
}
```

Transformation Layer

Data enrichment using mutate and filter plugins

Handles schema normalization for Elasticsearch

Adds metadata fields for pipeline tracking

Supports conditional processing for different data types

Filter: Example

```
filter {
}
    mutate {
        remove_field => ["date", "@timestamp", "host"]
    }

    json {
        source => "first_name"
        target => "name"
    }
}
```

Elasticsearch Output

BULK INDEXING FOR HIGH THROUGHPUT

CUSTOM INDEX NAMING STRATEGIES

DOCUMENT ID ENSURES IDEMPOTENT UPDATES

SUPPORTS NEAR REAL-TIME INDEXING

Output - Example

```
output {
    stdout { codec => json_lines }
    elasticsearch {
        hosts => ["http://localhost:9200"]
        index => "my_table_index"
        custom_headers => {
            "Authorization" => "${AUTH_KEY}"
        }
        document_id => "%{table_id}" # Unique identifier from the table
        timeout => 120
    }
}
```

Pipeline Execution:

- bin/logstash -f c:/logstash/sample_pipeline.conf

```
[2025-06-25T15:41:01,971][INFO ][logstash.inputs.jdbc     ][main][a7de5a26dd1108ebd5c360358f36fee9d1d91f7a04501cc6b87a56
b3d663c625] (0.098211s) SELECT CAST(current_setting('server_version_num') AS integer) AS v
[2025-06-25T15:41:02,064][INFO ][logstash.inputs.jdbc     ][main][a7de5a26dd1108ebd5c360358f36fee9d1d91f7a04501cc6b87a56
b3d663c625] (0.081702s) SELECT count(*) AS "count" FROM (SELECT * FROM talent.testing WHERE updated_dt > '1970-01-01 00:
00:00.000000+0000') AS "t1" LIMIT 1
[2025-06-25T15:41:02,149][INFO ][logstash.inputs.jdbc     ][main][a7de5a26dd1108ebd5c360358f36fee9d1d91f7a04501cc6b87a56
b3d663c625] (0.080756s) SELECT * FROM (SELECT * FROM talent.testing WHERE updated_dt > '1970-01-01 00:00:00.000000+0000'
) AS "t1" LIMIT 20000 OFFSET 0
{"first_name":"James","id":1,"last_name":"Smith","updated_dt":"2024-12-12T16:10:57.349Z","date":"2024-12-12T10:10:57.349
Z","@version":"1","@timestamp":"2025-06-25T20:41:02.167442600Z"}
{"first_name":"Kate","id":3,"last_name":"Williams","updated_dt":"2024-12-12T16:10:57.349Z","date":"2024-12-12T10:10:57.3
49Z","@version":"1","@timestamp":"2025-06-25T20:41:02.170098800Z"}
{"first_name":"John","id":2,"last_name":"Doe","updated_dt":"2024-12-12T16:10:57.349Z","date":"2024-12-12T10:10:57.349Z",
"@version":"1","@timestamp":"2025-06-25T20:41:02.169021400Z"}
```

Output – ES Index


```
{
    "_index": "testing",
    "_id": "1",
    "_score": 1.0,
    "_source": {
        "name": "James",
        "id": 1,
        "last_name": "Smith",
        "updated_dt": "2024-12-12T16:10:57.349Z",
        "@version": "1",
        "@timestamp": "2025-06-25T20:41:02.167442600Z"
    }
},
{
    "_index": "testing",
    "_id": "2",
    "_score": 1.0,
    "_source": {
        "name": "John",
        "id": 2,
        "last_name": "Doe",
        "updated_dt": "2024-12-12T16:10:57.349Z",
        "@version": "1",
        "@timestamp": "2025-06-25T20:41:02.169021400Z"
    }
}
```

Incremental Processing Strategy

- Uses tracking column (updated_at/timestamp)
- Avoids full dataset reload
- Reduces processing time and infrastructure cost
- Ensures only new or updated records are processed

Failure Handling Strategy

- Ensures pipeline resiliency and fault tolerance
- Prevents data loss during system failures
- Supports retry, buffering, and error isolation
- Maintains consistency across distributed systems

Input Failure Handling

- JDBC retry on connection issues
- Scheduling ensures automatic reprocessing
- Checkpointing via sql_last_value ensures recovery
- No need for full reload after failure

Processing & Data Errors

- Dead Letter Queue (DLQ) captures failed records
- tag_on_failure enables error identification
- Conditional routing isolates bad data
- Prevents pipeline disruption due to malformed input

Output Failure Handling

- Elasticsearch retry with exponential backoff
- Persistent queues buffer events during downtime
- Ensures eventual delivery
- Supports high availability scenarios

Data Consistency

USE DOCUMENT_ID FOR IDEMPOTENT WRITES

PREVENTS DUPLICATE CANDIDATE RECORDS

ENSURES EVENTUAL CONSISTENCY ACROSS SYSTEMS

CRITICAL FOR SEARCH ACCURACY AND ANALYTICS CORRECTNESS

Why Logstash for ETL

- Simpler setup compared to Spark and Kafka
- No cluster management required
- Configuration-driven pipeline design
- Faster time-to-market for data pipelines
- Ideal for real-time ingestion use cases

Cloud Deployment (AWS Example)

- Amazon RDS (PostgreSQL) as source
- Logstash deployed on EC2 or ECS
- Amazon OpenSearch/Self-managed ElasticSearch for indexing
- Kibana for visualization
- Scalable and cost-efficient deployment model

Performance Optimization

- Tune pipeline.workers for parallelism
- Adjust batch.size for throughput
- Optimize JDBC fetch size
- Use bulk indexing in Elasticsearch
- Monitor pipeline metrics for bottlenecks

Conclusion

LOGSTASH PROVIDES A LIGHTWEIGHT, REAL-TIME ETL SOLUTION

REDUCES INFRASTRUCTURE COMPLEXITY COMPARED TO ALTERNATIVES

ENABLES SCALABLE CANDIDATE SEARCH AND ANALYTICS

BUILT-IN RELIABILITY AND FAILURE HANDLING

IDEAL FOR EVENT-DRIVEN DATA ARCHITECTURES

THANK YOU!

Q&A